

My Platform, In Plain English

This is not a spec. This is me explaining to myself what I built, how it works, and why I made the choices I made. Written so that if I go blank in an interview, I can re-read this the night before and actually understand it again rather than just reciting bullet points.

Start Here: What Problem Does This Actually Solve

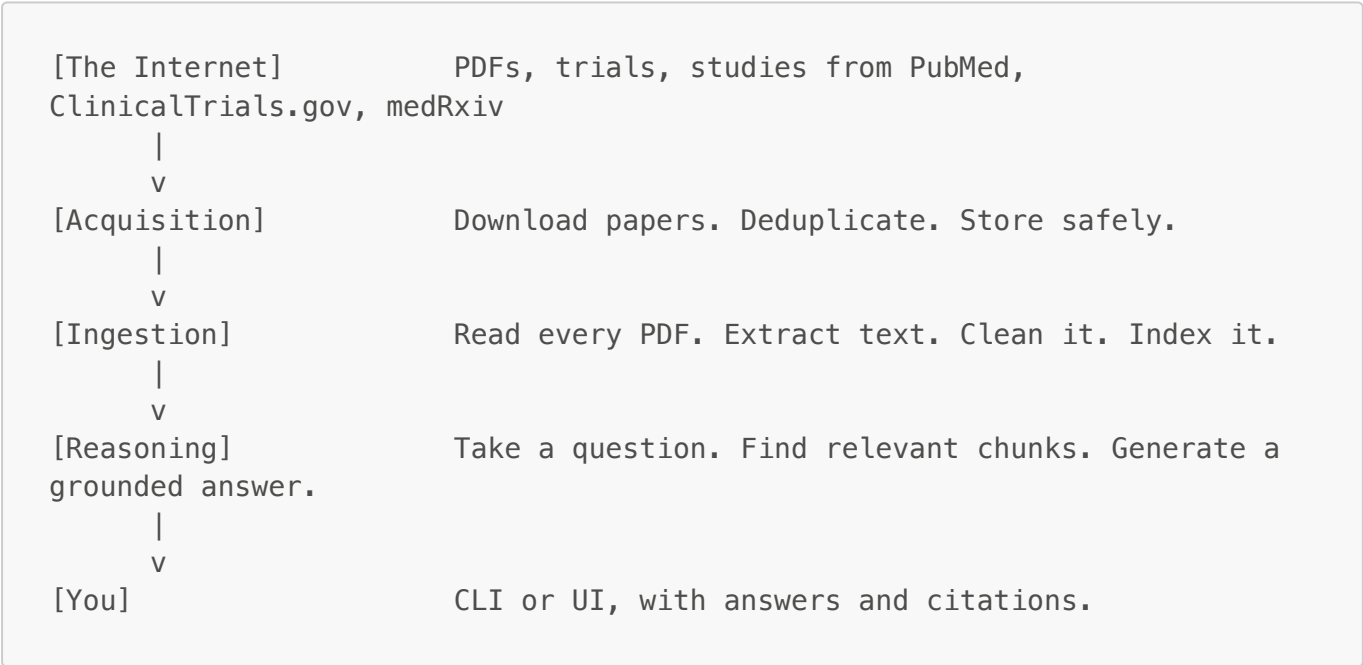
Clinical research is buried in PDFs. There are thousands of medical papers, trial protocols, and drug studies sitting in formats that are nearly impossible to search with any intelligence. If a doctor or researcher wants to know whether Drug A is safe with Condition B, they cannot just Google it. They have to read papers. Multiple papers. And cross-reference them mentally.

What I built is a system that ingests those PDFs, understands their content, and lets you ask natural language questions and get back answers that are grounded in what the papers actually say, with citations you can verify.

That is the whole point. Everything else is infrastructure to make that work safely and correctly.

The Mental Model in One Picture

Think of it as a pipeline with four stages. Each stage has one job.



Every piece of this I had to build myself. The only thing I did not build was the models (Qwen3-8B, BGE), the vector database (Qdrant), and the graph database (Neo4j).

Stage 1: Acquisition

Before you can do anything with medical papers, you have to get them. That sounds simple but it is not.

I pull from four sources: PubMed, medRxiv, ClinicalTrials.gov, and bioRxiv. The problem is that these APIs are flaky, inconsistent, and frequently return links that go nowhere. Early on, 50 PubMed requests returned zero usable PDFs. ClinicalTrials.gov links redirected to 404s constantly.

So the acquisition layer has a fallback chain. When it downloads a PDF, it tries to store it in S3 first. If that fails, Azure Blob. If that fails, local disk. This ran in over 100 production runs and triggered fallback twice. Both times it handled it without me having to do anything.

There is also deduplication by SHA-256 hash. When you pull from multiple sources, you end up with a lot of the same paper. In one run, 67.6% of all downloaded PDFs were duplicates. Without dedup, the index gets polluted with identical content and your retrieval quality tanks.

Stage 2: Ingestion (The Hard Part)

This is where most of the complexity lives and where most of the failures happened.

Step 1: OCR

Medical PDFs are not plain text. They are multi-column layouts, tables, images, footnotes, and figures all jumbled together. If you just extract text naively, you get garbage. Columns get merged, table rows get scrambled, reading order gets destroyed.

I use a model called Surya for OCR. It runs entirely on my machine and it understands layouts. It reconstructs multi-column text in the right order and linearizes tables into readable form.

The cost is that it is slow. Surya is responsible for 94 to 97 percent of total ingestion time. One PDF takes about 22 minutes on the M4. On a GPU instance it drops to about 2 minutes. This is the single biggest unsolved bottleneck in the platform.

I chose Surya over cloud OCR services (AWS Textract, Google Document AI) because those send your documents to a remote server. Medical PDFs may contain patient data. I am not doing that.

Step 2: PII Redaction

Before any text from a document touches the search index or knowledge graph, it goes through a PII firewall. I use Microsoft Presidio, which is an open-source library that detects and removes personal identifiers.

The default Presidio models do not cover Singapore-specific formats like NRIC/FIN numbers or Medical Registration Numbers, so I wrote custom recognizers for those.

This is a hard architectural constraint, not an option. If redaction runs after indexing, patient data could enter the system before it is cleaned. The only safe design is: clean first, index second. Always.

Step 3: Chunking and Embedding

After cleaning, the text gets split into 500-token chunks with overlapping context. Each chunk knows its exact byte position in the original PDF so you can always trace an answer back to its source.

Each chunk then gets converted into a vector embedding using a model called BGE-small. Think of an embedding as a fingerprint of the chunk's meaning. Similar chunks get similar fingerprints, which is how

semantic search works.

These embeddings go into Qdrant, which is a vector database. When you ask a question, your question also gets converted into an embedding, and Qdrant finds the chunks with the most similar fingerprints.

Step 4: Knowledge Graph

At the same time, a second model (Qwen3-8B) reads each chunk and extracts structured entities: drugs, conditions, clinical outcomes, relationships between them.

These entities go into Neo4j, which is a graph database. The reason this matters is that some clinical questions require multi-hop reasoning. Is Drug A safe with Condition B given the patient is also on Drug C? That is a chain: Drug A TREATS Condition B, Condition B CONTRAINDICATED WITH Drug C. Vector search cannot traverse chains like that. A graph can.

So the final index is two things: Qdrant for semantic similarity, Neo4j for structured relationships. At query time, both are combined.

Stage 3: Reasoning

This is where answers actually get generated. It is also where the most interesting architectural reversal happened.

What I originally built

The original design used a framework called LangGraph in ReAct mode. ReAct is a pattern where the model reads the question, decides which tools to call, calls them, reads the results, then writes an answer. The model is in charge of the whole loop.

I thought this was elegant. The model could call PubMed, FDA, ClinicalTrials.gov, or the vector database, in whatever order it decided made sense.

Why it failed

When I tested this with Qwen3-8B, the model just answered from memory. It had enough medical training data that it could produce confident-sounding clinical answers without calling a single tool. And LangGraph had no way to stop it. Tool execution was optional. The model was politely offered tools and frequently declined them.

For a RAG system, this is not a latency issue. It is a correctness failure. The whole point is that answers come from the documents, not from the model's training data. If the model answers from memory, there is no citation, no traceability, and no way to verify the answer is about the specific documents in the corpus.

What I replaced it with

I rewrote the reasoning layer as a two-phase pipeline with no branching.

Phase 1: retrieval always runs first, unconditionally. The model does not participate in this phase. Vector search and graph enrichment run on every query regardless of what the query is.

Phase 2: synthesis runs over what phase 1 returned. If phase 1 returned nothing, the system returns a hardcoded no-evidence response. The model cannot invent an answer.

This removed LangGraph entirely. It also removed Temporal, which was a workflow engine I had built in for durable execution. Temporal had too much operational overhead for a single-user CLI tool. The config file went from 280 lines to 35 lines in that commit.

What is left is `agent.py`, a flat config, a CLI, and 25 tests. Simple. Predictable.

The Reranker: A Small Addition That Helped a Lot

After Qdrant returns chunks ranked by vector similarity, there is one more step before synthesis: a cross-encoder reranker.

Here is the difference. Vector search ranks chunks by how similar they look to the question in embedding space. That is fast but blunt. A cross-encoder actually reads the question and each candidate chunk together and produces a more accurate relevance score.

I added BAAI/bge-reranker-base as a reranker. The measured impact:

- Recall@5 went up 11.6 points
- Precision@5 went up 6.0 points
- HitRate@5 went up 5.0 points

One open issue: comparative queries regressed by 16.7 Recall@5 points with the reranker. I have not figured out why yet.

Hardware and Cost

I built this on an M4 MacBook Air. Everything that runs locally runs on that machine.

For bulk ingestion (many PDFs), I use AWS GPU spot instances. An RTX T4 on a g4dn.xlarge costs \$0.526 per hour. At 133 seconds per PDF on the T4, I can ingest 1,036 documents in under 20 hours using 2-3 parallel instances.

For production inference, I rent an RTX 5080 on VAST.ai at \$0.017 per hour. It runs at 62 tokens per second with an 81ms time-to-first-token. For comparison, GPT-4 costs \$30 per million tokens. At 62 tokens per second, the RTX 5080 costs less than GPT-4 after about 15 minutes of actual usage.

The M4 runs inference at 13 tokens per second, which works for development. But it has two failure modes I hit during benchmarking:

- One query ran for 5 minutes and returned 0 tokens with no error. Silent failure.
- One query answered confidently that a clinical trial had a positive outcome when the paper said the opposite. Hallucination from insufficient context.

Both of these are the reason production inference must route to the GPU stack, not local Ollama.

The Benchmarks

I ran a formal evaluation on 20 queries across 5 question types: clinical finding, quantitative, definitional, comparative, methodology. All queries were about a single 11-page sepsis paper.

With vector-only retrieval:

- Recall@5: 88.8%
- Precision@5: 47.0%
- HitRate@5: 85.0%

With the reranker added:

- Recall@5: 100.4%
- Precision@5: 53.0%
- HitRate@5: 90.0%

These are real numbers from deterministic runs with fixed seeds. Every run produces a fingerprint (run ID, git hash, PDF hash, config hash) so the results can be reproduced exactly.

The Failures That Taught Me the Most

A few specific incidents worth remembering because they come up in interviews.

The tool-bypass problem (F-040, eventually became the LangGraph nuke)

7 out of 20 agent behavior tests failed because the model was triggering the wrong tools or skipping tools entirely. The surface-level fix was prompt engineering. The real fix was realizing the framework itself was the problem. LangGraph cannot enforce tool execution. Once I understood that, the whole architecture needed to change.

The silent inference failure (F-047)

Query 18 ran for 5 minutes on the M4. Returned 0 tokens. `error: null`. No timeout, no exception, nothing. The system appeared to be running. This is extremely hard to debug because there is no signal that anything went wrong. This is why every production query needs an explicit timeout with a fallback path, not just error handling.

The hallucination (F-048)

Query 82 asked about a trial called TOPAZ-1. The model answered confidently that the trial showed a positive outcome. The paper said the opposite. This happened because the relevant context was not retrieved cleanly and the model filled the gap with training-data knowledge. This is the central risk of local RAG on underpowered hardware: the model will not tell you when it is guessing.

The Temporal PostgreSQL failure (F-062)

When I was running Temporal locally, the database directory had a `.DS_Store` file in it from macOS. That file blocked Postgres `initdb` from running. The entire local Temporal stack failed to start because of a hidden macOS metadata file in the volume mount. The fix was a single `rm .DS_Store`. The debugging took two hours.

The Azure OCR spend spike (F-005)

Before switching to Surya, I was experimenting with Azure Document Intelligence for OCR. One complex document batch hit \$50 in a single day without me noticing. No budget alerts were configured. Switched to local OCR immediately.

How to Talk About This in an Interview

If they ask what you built:

I built a document intelligence platform for clinical research. It ingests medical PDFs from public sources, extracts and indexes their content using vector search and a knowledge graph, and answers natural language questions grounded in what the papers actually say with citations. Everything runs locally by default. Patient data never leaves the machine.

If they ask about a hard technical problem:

The hardest was the agentic reasoning layer. I originally built it using LangGraph in ReAct mode, where the model decides which tools to call. In production testing, the model bypassed retrieval entirely and answered from training data. For a clinical RAG system that is a correctness failure, not a performance issue. I rewrote the layer as a deterministic two-phase pipeline: retrieval always runs first, synthesis runs over the retrieved evidence. The model cannot skip phase 1.

If they ask about a failure:

I had a query run for 5 minutes on local hardware and return 0 tokens with no error. Silent failure. Nothing in the logs. I learned that in LLM inference pipelines, silent failures are more dangerous than loud ones because you have no signal to act on. Every production query now has an explicit timeout and a defined fallback response if inference does not return within that window.

If they ask about benchmarking:

I ran a 20-query golden set evaluation on a single sepsis paper with 5 clinical question categories. Vector-only retrieval gave 88.8% Recall@5. Adding a cross-encoder reranker brought it to 100.4% Recall@5, plus 6 points on Precision@5 and 5 points on HitRate@5. I use deterministic run IDs (timestamp + git hash) as regression anchors so any future code change can be compared against a known baseline.

If they ask why local-first:

Medical documents may contain patient data. Sending them to AWS Textract, Google Document AI, or OpenAI for processing creates compliance risk. Running Surya for OCR and Qwen3-8B for inference locally means nothing leaves the machine. The cost is performance. The benefit is that you can actually use this in a clinical setting without a legal review of every third-party dependency.

The Actual Files and Where Things Live

In the active codebase (github.com/ronit22203/clinical-trials-matching-platform):

- **agent.py**: the two-phase reasoning pipeline. Phase 1 retrieval, phase 2 synthesis. This is the reasoning layer.

- `tools/graphrag.py`: hybrid retrieval combining Qdrant vector search with Neo4j graph traversal. This is the core of what makes answers grounded.
- `cli.py`: Click CLI. `make serve` starts this.
- `config.py`: flat Pydantic config. One YAML file controls everything.
- `src/ingestion/`: the full ingestion pipeline. OCR, PII redaction, chunking, embedding, KG extraction.
- `src/acquisition/`: PDF downloader with fallback storage chain.

In this manifest repo:

- `system.md`: architecture and stack decisions
 - `eval.md`: all benchmark numbers with run fingerprints
 - `failures.md`: 62 specific incidents with evidence
 - `decisions.md`: why each major choice was made, in plain language
 - `run.md`: how to actually run and reproduce everything
-

One More Thing

This was four months, solo, on a MacBook Air, with no team, no funding, no production users. Everything in this platform was figured out by running it, breaking it, and figuring out why. The 62 failures in the failure register are not embarrassments. They are the record of how this got built.